

# BÁO CÁO TỔNG KẾT ĐỀ TÀI

## *Hệ thống Laser Điều hướng 2 Trục XY*

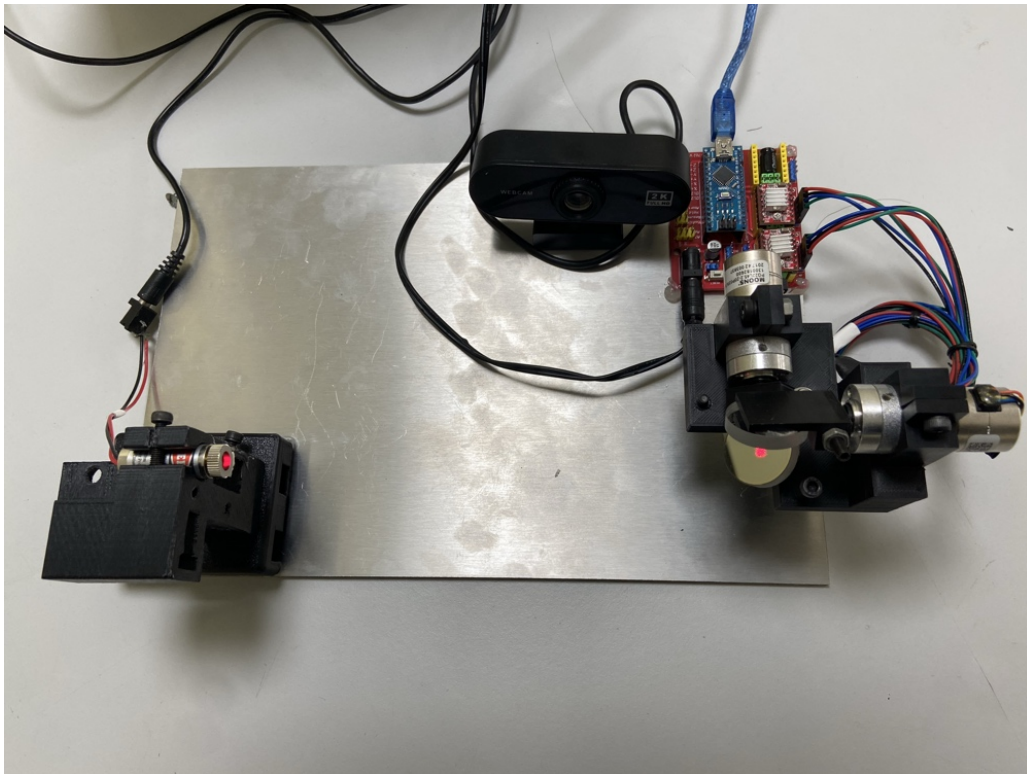
### 1 Mục tiêu và Tổng quan hệ thống

Đề tài tập trung xây dựng một hệ thống điều khiển tia laser định hướng linh hoạt trong không gian 2D. Tia laser được chiếu qua 2 gương phẳng gắn trên 2 động cơ bước (trục X và trục Y). Hệ thống nhận lệnh tọa độ từ máy tính, điều khiển động cơ quay chính xác để tia laser hướng tới vị trí mong muốn.

Điểm khác biệt của hệ thống là khả năng **tự động hiệu chuẩn (Auto-calibration)** và **nội suy tọa độ qua Camera**, giúp người dùng chỉ cần click chuột lên hình ảnh là tia laser sẽ tự động bắn tới đúng vị trí thực tế trên tường.

### 2 Phần cứng đã hoàn thiện

- **Khối cơ khí:** Đã lắp đặt cố định module phát laser, camera, bo mạch điều khiển, ngàm giữ động cơ và gương phản xạ.
- **Khối điều khiển trung tâm:** Sử dụng bo mạch **Arduino Nano** gắn trên shield mở rộng **CNC Shield V4**, kết hợp cùng 2 module driver điều khiển động cơ **A4988**.
- **Kết quả phần cứng:** Hệ thống đã kết nối thành công với máy tính qua cổng Serial, động cơ chạy ổn định. Thông số đo đạc thực tế: động cơ cần **904 bước (step)** để quay hết 1 vòng  $360^\circ$  (ở chế độ full-step).



Hình 1: Cấu trúc tổng thể của Hệ thống Laser Điều hướng sau khi tích hợp

#### **Lưu ý kỹ thuật về phần cứng (Lỗi thiết kế mạch CNC Shield V4):**

Trong quá trình thực hiện, một vấn đề đã được phát hiện là mạch CNC Shield V4 có lỗi thiết kế từ nhà sản xuất: các chân cầu hình vi bước (MS1, MS2, MS3) không được nối với nguồn 5V. Do đó, dù có cắm Jumper, driver A4988 vẫn bị ép chạy ở chế độ Full-step gây ồn và rung.

**Giải pháp khắc phục:** Cần can thiệp phần cứng (hàn cầu dây 5V vào các chân MS dưới gầm mạch CNC V4) để kích hoạt chế độ vi bước (1/16 step), giúp tia laser di chuyển mịn màng hơn.

## **3 Phần mềm và Thuật toán điều khiển**

### **3.1 Firmware trên Arduino (C++)**

Bo mạch Arduino đóng vai trò là thiết bị chấp hành (Slave), có nhiệm vụ chuyển đổi các lệnh điều hướng từ máy tính thành tín hiệu xung điện điều khiển động cơ.

- **Quản lý chuyển động:** Firmware được phát triển bằng C++.

Hệ thống tích hợp hai thư viện AccelStepper, MultiStepper. Các thư viện này cung cấp thuật toán nội suy gia tốc, giúp động cơ khởi động và dừng lại mượt mà, tránh hiện tượng trượt bước (missed steps) do thay đổi vận tốc đột ngột.

- **Giao thức truyền thông:** Vòng lặp chính (loop) liên tục đọc dữ liệu từ cổng Serial ở baudrate 115200. Khi nhận được chuỗi tọa độ định dạng “X,Y”, Arduino sẽ tách chuỗi, chuyển đổi sang

số nguyên và ra lệnh cho động cơ di chuyển đồng bộ tới tọa độ đích. Sau khi cả hai trục X và Y đã hoàn tất hành trình (`distanceToGo() == 0`), Arduino gửi phản hồi “DONE” về máy tính để đồng bộ hóa các bước tính toán tiếp theo. Ngoài ra, lệnh “r” được cấu hình riêng để đưa hệ thống về gốc tọa độ giả định.

## 3.2 Phần mềm trung tâm trên Máy tính (Python + OpenCV)

Phần mềm trung tâm trên máy tính được xây dựng bằng Python, sử dụng thư viện OpenCV để xử lý luồng video trực tiếp từ camera.

### 3.2.1 Thuật toán nhận diện và lọc nhiễu tia laser

Mục tiêu là xác định chính xác tọa độ  $(X, Y)$  của đốm laser trên màn hình trong thời gian thực. Hàm xử lý ảnh `tim_toa_do_laser()` được triển khai qua các bước:

1. **Chuyển đổi không gian màu:** Khung hình từ camera được chuyển từ hệ màu BGR sang ảnh xám (Grayscale).
2. **Lọc ngưỡng (Thresholding):** Áp dụng ngưỡng sáng mức 245 để tách riêng các điểm chói sáng ra khỏi nền tối.
3. **Nhận diện đường viền (Contours):** Sử dụng hàm `cv2.findContours` để trích xuất các mảng sáng.
4. **Tối ưu và lọc nhiễu:** Một cải tiến quan trọng đã được áp dụng là sắp xếp danh sách các mảng sáng theo diện tích giảm dần. Thay vì chỉ lấy mảng sáng to nhất (có thể là ánh đèn phản chiếu hoặc vùng lóa sáng), thuật toán duyệt qua từng mảng và áp dụng bộ lọc hình học kép:
  - *Điều kiện diện tích:* Chỉ chấp nhận các đốm sáng có diện tích nằm trong khoảng từ 10 đến 400 pixel.
  - *Điều kiện độ tròn:* Tính toán độ tròn dựa trên chu vi và diện tích ( $4\pi \cdot S/C^2$ ). Đốm sáng được xác nhận là laser chỉ khi độ tròn đạt mức  $> 0.75$ .

Tọa độ tâm của hình tròn ngoại tiếp (`minEnclosingCircle`) viền laser đạt chuẩn sẽ được trả về làm kết quả nhận diện.

### 3.2.2 Nghiên cứu và cải tiến thuật toán điều hướng

Quá trình nghiên cứu và tối ưu thuật toán trong kỳ thực tập được chia thành hai giai đoạn phát triển chính.

#### Giai đoạn 1: Tiếp cận bằng ánh xạ tuyến tính và bù rơ cơ khí

Ban đầu, hệ thống được thiết kế theo phương pháp quy đổi tuyến tính. Thuật toán điều khiển động cơ nhích một số bước nhất định (BUOC\_CALIB), đo khoảng cách pixel di chuyển được trên camera, từ đó tính ra tỷ lệ quy đổi (Pixel/Step) cho từng trục X và Y. Trong quá trình này, hiện tượng rơ cơ khí

(Backlash) của bánh răng và dây đai đã làm sai lệch tọa độ. Để giải quyết, một cỗ máy trạng thái (State Machine) phức tạp đã được lập trình để tự động chạy tiến, lùi, và chạy chéo nhằm đo lường chính xác lượng bước hao hụt. Các thông số rơ (backlash\_X, backlash\_Y) được lưu lại và tự động cộng dồn vào lệnh điều khiển khi động cơ đảo chiều.

*Hạn chế phát sinh:* Dù đã bù rơ thành công, thuật toán gặp thất bại lớn khi áp dụng ở góc chiếu xiên. Sự không đồng trục giữa camera và laser gây ra hiện tượng méo phối cảnh (Perspective Distortion). Tỷ lệ quy đổi (Pixel/Step) không còn là một hằng số tuyến tính, dẫn đến sai số nghiêm trọng khi điều hướng tia laser ở các vùng biên của mục tiêu.

## Giai đoạn 2: Tối ưu bằng Ma trận Homography và RANSAC

Để khắc phục bài toán méo phối cảnh, toàn bộ thuật toán tuyến tính và bù rơ thủ công đã được thay thế bằng phương pháp không gian hình học xạ ảnh (Projective Geometry). Công việc triển khai bao gồm:

- **Tự động quét lưới (Grid Calibration):** Hệ thống tự động sinh ra một mảng lưới tọa độ step (ví dụ: lưới  $5 \times 5$ ). Động cơ sẽ lần lượt di chuyển tới các điểm này, dừng lại ổn định (Stabilizing) để camera chụp và ghi nhận tọa độ pixel thực tế của điểm laser.
- **Tính toán Homography:** Sử dụng hàm `cv2.findHomography` để tìm ra mối quan hệ biến hình không gian giữa tọa độ điểm ảnh (Source) và tọa độ động cơ (Destination). Thuật toán RANSAC được tích hợp đồng thời để tự động loại bỏ các điểm dữ liệu nhiễu (outliers) sinh ra do sai số cơ khí cục bộ.
- **Nội suy tọa độ vô tận:** Sau khi quá trình Calib hoàn tất, hệ thống thu được một “Ma trận phối cảnh” lõi. Khi người dùng nhấp chuột vào bất kỳ vị trí nào trên luồng video, phần mềm sử dụng hàm `cv2.perspectiveTransform` để nhân tọa độ pixel với ma trận này. Kết quả trả về là số bước X, Y chính xác tuyệt đối mà động cơ cần quay, tự động triệt tiêu mọi sai số méo hình và độ lệch trục vật lý mà không cần dùng đến các phương trình lượng giác phức tạp.

### 3.2.3 Thiết kế máy trạng thái và tích hợp tổng thể

Hệ thống phần mềm cuối cùng được tổ chức dưới dạng Máy trạng thái hữu hạn (Finite State Machine) vận hành trong vòng lặp vô hạn của OpenCV. Các trạng thái được định nghĩa rõ ràng bao gồm: Khởi động quét lưới (START\_GRID\_CALIB), chờ phần cứng ổn định (STABILIZING), đọc dữ liệu (READ\_GRID\_CALIB), tính toán toán học (CALCULATE\_HOMOGRAPHY), và sẵn sàng chờ lệnh người dùng (READY).

Đồng thời, hàm xử lý sự kiện nhấp chuột (Mouse Callback) được tích hợp vào giao diện OpenCV. Hệ thống tự động hiển thị các chỉ báo UI (đường chéo mục tiêu, khung tròn báo hiệu vị trí laser, thông báo trạng thái hiện tại) giúp người dùng dễ dàng theo dõi và tương tác với quá trình điều khiển một cách trực quan.

## 4 Các vấn đề tồn đọng và Hướng phát triển tiếp theo

### 4.1 Phân tích nguyên nhân và Đề xuất cách khắc phục

Hệ thống bộc lộ hai dạng sai số đặc trưng: sai lệch nhỏ ngẫu nhiên ngay trong vùng trung tâm và độ lệch tâm rõ rệt tại các mép/góc khung hình. Đây là các hiện tượng vật lý - toán học rất phổ biến trong các hệ thống thị giác máy tính tích hợp cơ khí. Qua đối chiếu thực nghiệm, các nguyên nhân cốt lõi được xác định bao gồm:

#### 1. Sai số do làm tròn độ phân giải phần cứng (Quantization Error):

- *Phân tích:* Đây là nguyên nhân chính dẫn đến các sai số nhỏ ngẫu nhiên ở Kịch bản 1. Do mạch CNC Shield V4 gặp lỗi thiếu nguồn 5V, động cơ bắt buộc phải chạy ở chế độ Full-step (bước to). Khi thuật toán Homography tính toán nội suy và trả về kết quả là một số thực (Float), bộ điều khiển vi bước phải làm tròn thành số nguyên (Int) gần nhất. Sự làm tròn này khiến đốm laser thường bị nhảy vượt qua hoặc chưa vươn tới chính xác điểm vi chỉnh, làm đốm sáng thỉnh thoảng lệch ra mép của tâm ngắm.
- *Khắc phục:* Triển khai giải pháp can thiệp hàn mạch đã đề xuất ở Chương 2 để kích hoạt chế độ vi bước (1/16 step). Độ phân giải cơ khí tăng lên gấp 16 lần sẽ giúp tia laser chia nhỏ quãng đường, đáp ứng được các tọa độ lẻ và triệt tiêu gần như hoàn toàn sai số làm tròn này.

#### 2. Hiện tượng méo quang học của thấu kính (Radial Distortion):

- *Phân tích:* Nguyên nhân này giải thích trực tiếp cho sai số rõ rệt ở Kịch bản 2. Thuật toán Ma trận Homography hoạt động dựa trên giả định không gian phẳng tuyệt đối. Tuy nhiên, các camera thực tế luôn có độ cong thấu kính, gây ra méo hình (Barrel/Pincushion distortion) mạnh nhất ở các mép và góc ảnh. Đường thẳng trong không gian thực tế bị uốn cong trên ảnh pixel ở vùng biên, phá vỡ tính tuyến tính của phép chiếu, khiến ma trận Homography tính toán sai lệch góc quay của động cơ.
- *Khắc phục:* Cần bổ sung quy trình hiệu chuẩn camera (Camera Calibration) bằng thuật toán quét bàn cờ mẫu (Chessboard) để tìm ra ma trận biến dạng (Distortion Coefficients)[?], từ đó nắn phẳng luồng video gốc trước khi đưa vào khối xử lý tọa độ.

#### 3. Giới hạn ngoại suy của mô hình toán học (Extrapolation Error):

- *Phân tích:* Trong quá trình tự động lấy mẫu (START\_GRID\_CALIB), phần cứng chỉ di chuyển quét một lưới điểm giới hạn ở vùng không gian an toàn (thường nằm ở trung tâm). Khi người dùng ra lệnh ở khu vực 4 góc cực hạn (nằm ngoài vùng đã lấy mẫu), hệ thống toán học buộc phải thực hiện phép ngoại suy (Extrapolation). Mục tiêu càng xa vùng lưới cơ sở, sai số tuyến tính tích lũy càng lớn.
- *Khắc phục:* Tinh chỉnh lại tham số hành trình khởi động, mở rộng biên độ của vòng lặp quét lấy mẫu ra sát với giới hạn vật lý của cơ cấu cơ khí hơn để tăng độ bao phủ của ma trận gốc.

## 5 Mã nguồn và Hướng dẫn thực hành

Toàn bộ mã nguồn (bao gồm chương trình C++ cho Arduino và script Python cho máy tính), tài liệu hướng dẫn chi tiết về cách chạy hệ thống đã được đăng tải tại kho lưu trữ mã nguồn mở sau:

**Liên kết GitHub:** [https://github.com/LLC536/Laser\\_dieu\\_huong\\_VienVatLy](https://github.com/LLC536/Laser_dieu_huong_VienVatLy)